HW 2B

A secure login for a Juice Shop-style shop

Secure feature design, a login form with checks on both sides, and an attempt to break that form.

Part 1. Secure feature design

Juice Shop has three flaws this login closes. The login query is open to SQL injection: a crafted email can change the statement and skip the password check. Binding the email with WHERE email = ? keeps it as data, and values that are not emails are rejected first. Search and reviews are open to cross-site scripting: a script planted there runs in another shopper's browser. Writing output as text, plus a policy that blocks inline scripts, stops that. Registration fails when the check lives only on the page or the secret is weak. The server decides identity. Passwords use bcrypt.hashSync(password, 10), and bcrypt.compareSync checks a guess without reversing the hash.

Part 2. Login form

This is an HTML, CSS, and JavaScript login, plus account creation. validateLogin blocks an empty submit, requires @ in the email, and requires a password of at least 8 characters. The browser runs that check, and the server runs it again. Passwords are stored with bcrypt at cost 10. After sign-in, the customer can order drinks. Run npm install, then npm start, and open http://127.0.0.1:47291. The sample account is demo@juice.lab / Lemonade1. README.md has the same steps. Publish this folder on GitHub and paste that public URL here. This copy has no public link yet.

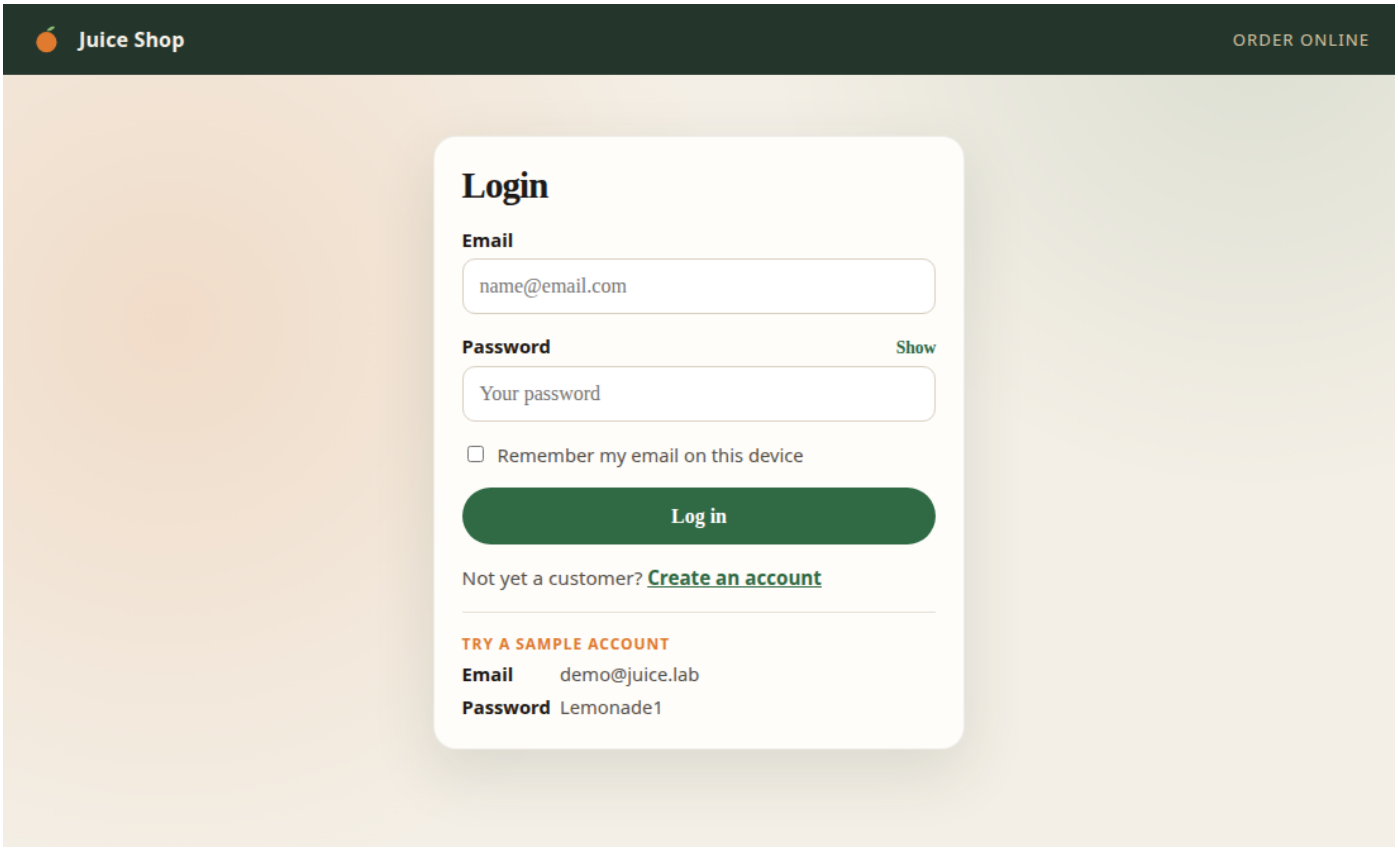


Figure 1. The sign-in card, with the sample account.

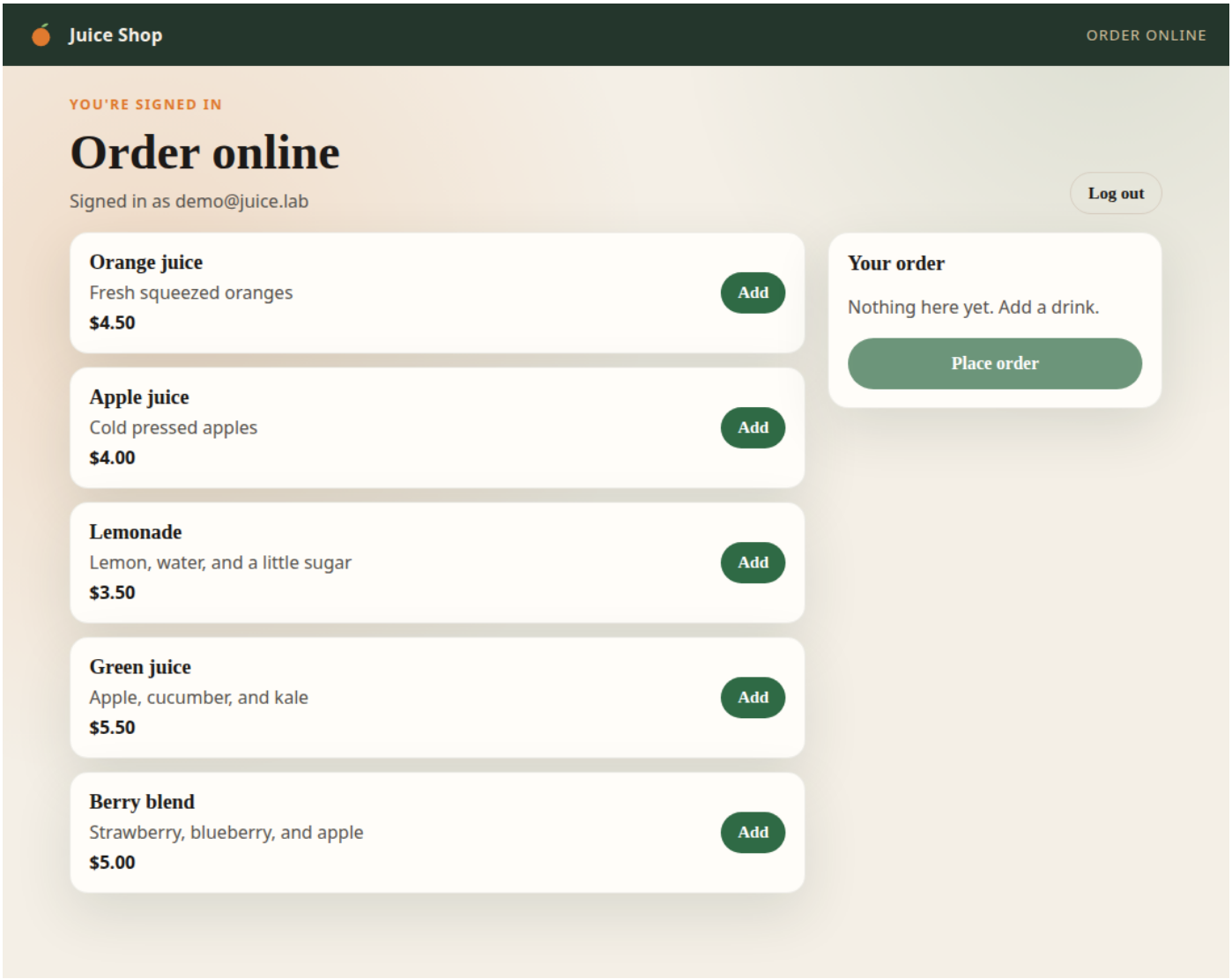


Figure 2. After sign-in, the customer can order drinks. Log out stays available.

Part 3. Trying to break the form

I tried to break this login at http://127.0.0.1:47291. An empty submit said the email and password are required. A short password was rejected for length. An email shaped like a database command was rejected with "That email or password doesn't match our records." The sample account stayed locked because the email is bound to ?. An email containing markup did not run a script. Those characters stayed in the box as text. The attack failed. The fix is the bound query and treating input as text. If that input were inserted as HTML, the browser could run it.

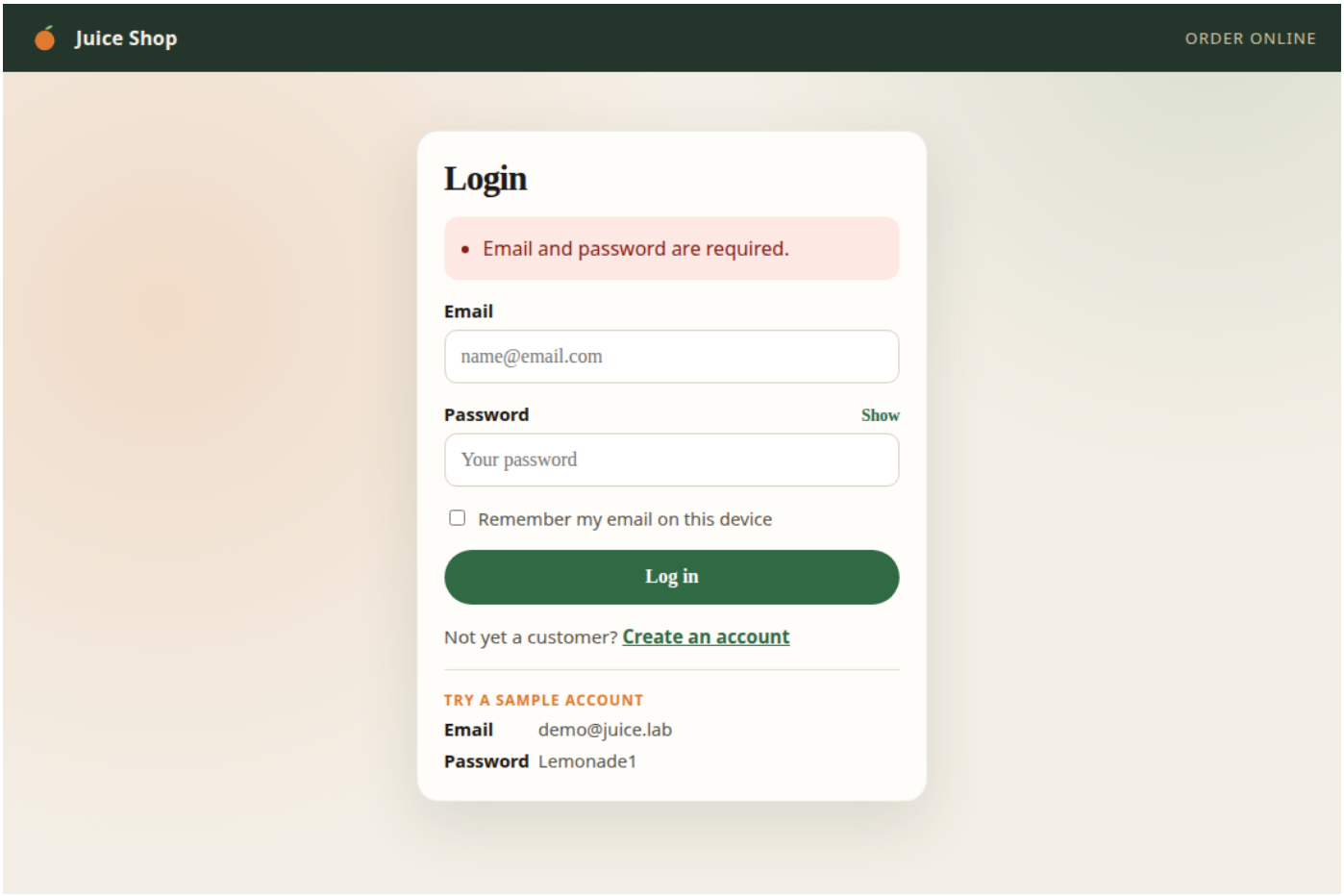


Figure 3. An empty submit is rejected.

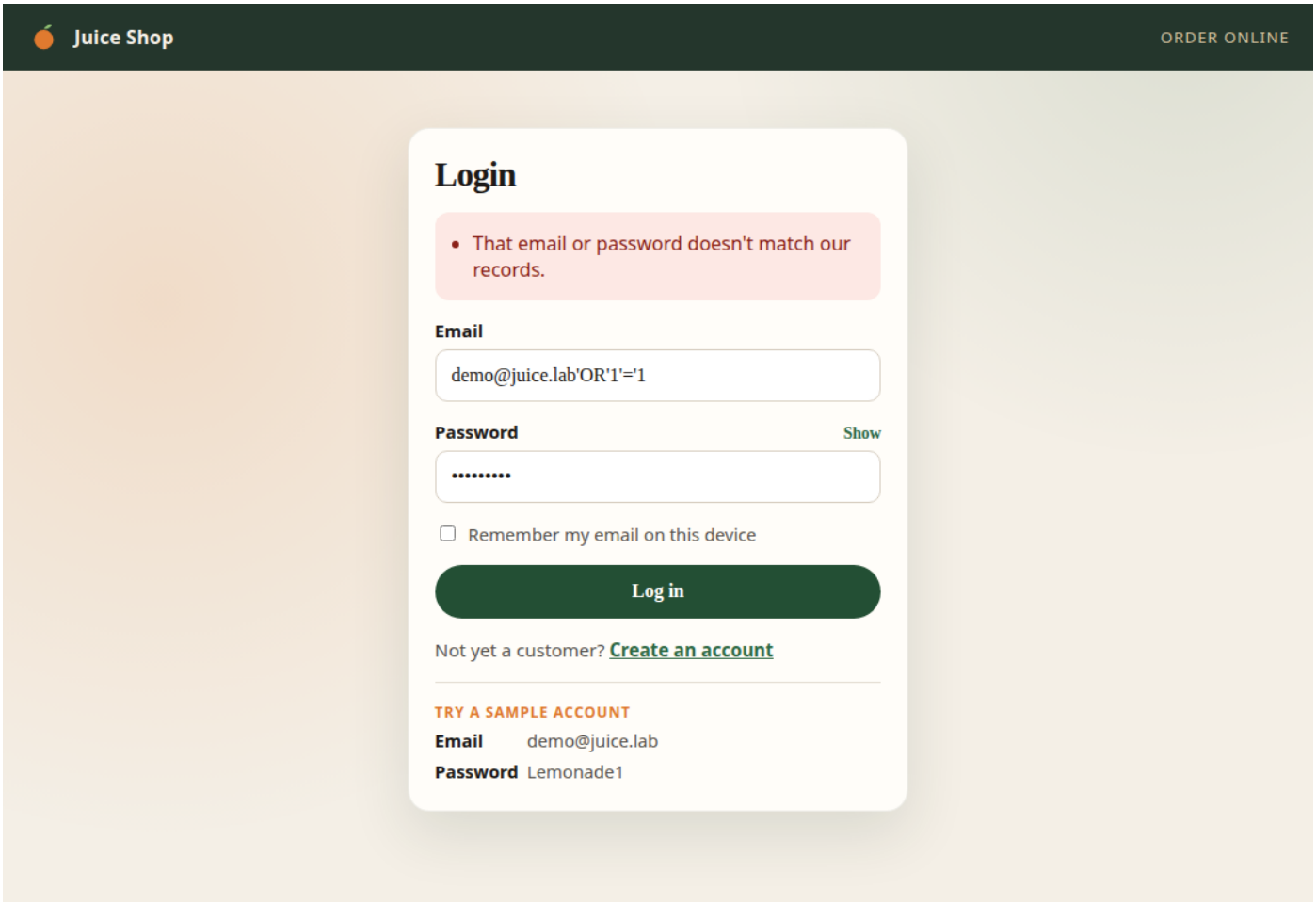


Figure 4. A database-shaped email is rejected. Sign-in does not succeed.

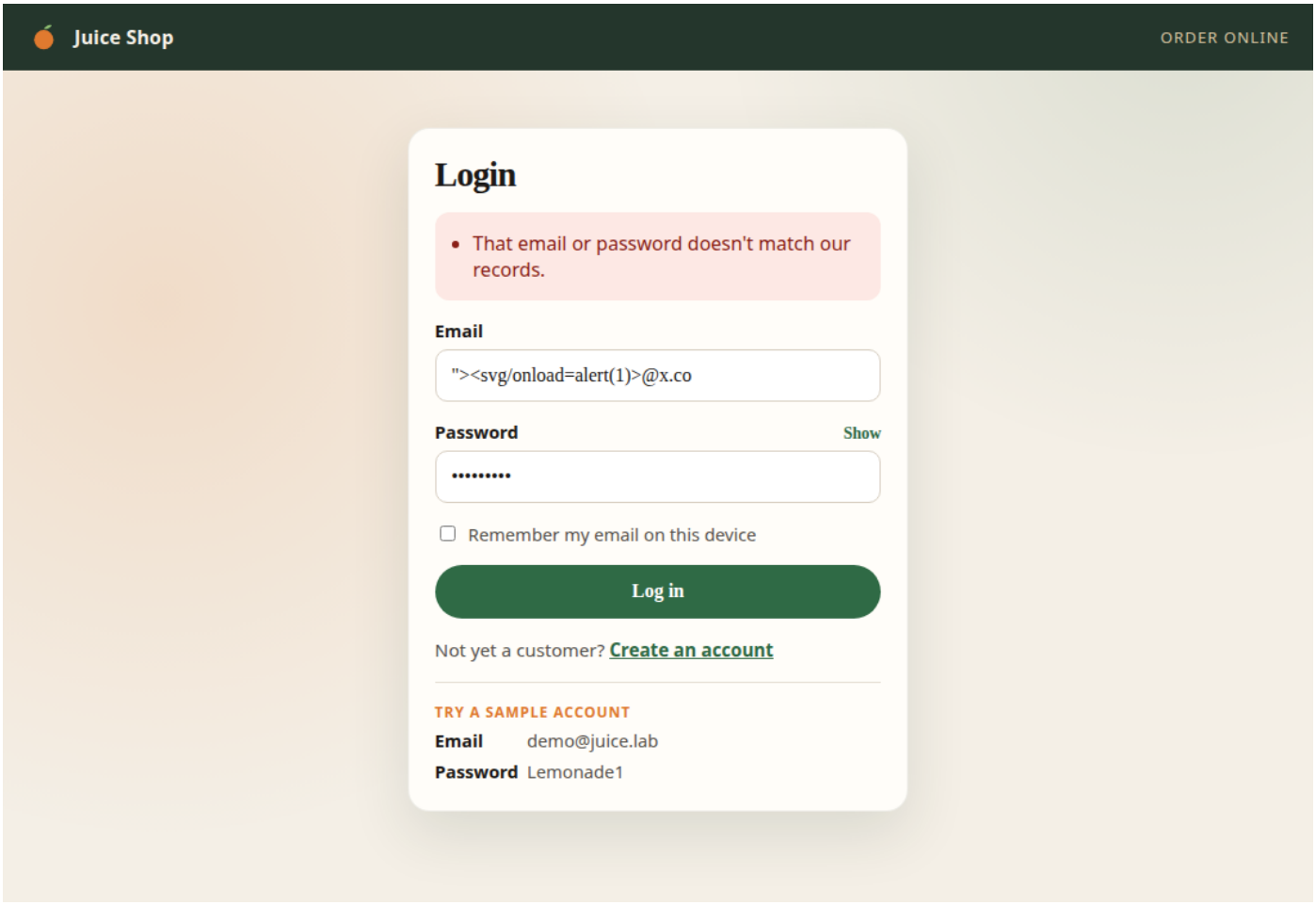


Figure 5. Markup in the email stays text. No script dialog ran.